

PYTHON CHEAT SHEET

1. BASICS

<code>x = 5 / x: int = 5</code>	assignment / typed
<code>type(x) len(x) print(x) input("msg")</code>	built-ins
<code>+ - * / // % **</code>	arithmetic
<code>== != > < >= <= and or not</code>	comparison & logical
<code>= += -= *= /= %=</code>	assignment operators

Types: `int float str bool ( True / False ) None` **F-string:** `f"{val:.2f}" f"{val:>10}" f"{val:,}" f"{name!r}"`

Conditionals: `if / elif / else` — **For:** `for i in range(start, stop, step)` — **While:** `while condition:` — **Break:** `break / continue`

2. DATA STRUCTURES

Structure	Syntax	Key Methods
List <code>[]</code>	mutable, ordered	<code>.append(x)</code> <code>.extend(lst)</code> <code>.insert(i,x)</code> <code>.remove(x)</code> <code>.pop(i)</code> <code>.sort()</code> <code>.reverse()</code> <code>.clear()</code> <code>.copy()</code>
Tuple <code>()</code>	immutable, ordered	<code>.count(x)</code> <code>.index(x)</code>
Dict <code>{k:v}</code>	mutable, key- value	<code>.get(k,default)</code> <code>.keys()</code> <code>.values()</code> <code>.items()</code> <code>.update(d)</code> <code>.pop(k)</code> <code>.clear()</code>
Set <code>{}</code>	mutable, unique	<code>.add(x)</code> <code>.update(lst)</code> <code>.remove(x)</code> <code>.discard(x)</code> <code>.pop()</code>

Slicing: `lst[start:stop:step]` `lst[-1]` `lst[::-1]` (reverse) **Swap:** `a, b = b, a` — **Unpack:**
`x, y, z = my_tuple` **Set ops:** `a | b` union · `a & b` intersect · `a - b` diff · `a ^ b` sym diff
Copy: `lst[:]` or `lst.copy()` (shallow) · `[i[:] for i in lst]` (deep) **Freq pattern:** `d[k] =`
`d.get(k, 0) + 1`

3. COMPREHENSIONS

TEXT

```
1 [x**2 for x in lst]           # basic
2 [x for x in lst if x > 0]     # filtered
3 [f(i) for i in range(n) if cond] # combined
4 {k: v for k, v in pairs}      # dict comp
5 {len(w) for w in words}      # set comp
```

4. FUNCTIONS

PYTHON

```
1 def name(param, default=val):    # default param
2     return expr
3
4 def f(*args): ...                # positional args (tuple)
5 def f(**kwargs): ...            # keyword args (dict)
```

Scope: `global x` (modify global) · `nonlocal x` (modify enclosing) **Lambda:** `lambda x, y: x + y`
HOF: `sorted(lst, key=lambda x: x["k"])` · `filter(fn, lst)` · `map(fn, lst)` · `reduce(fn, lst, init)`

Recursion: base case + recursive case (`factorial(0)=1` , `fib(0)=0` , `fib(1)=1`)

Decorator: `@decorator` — wraps function with extra behavior. Always use `@wraps(func)` inside.

Closure: inner function captures outer variable + `nonlocal`

5. ERROR HANDLING

PYTHON

```
1 try:
2     risky_code()
3 except ValueError as e:        # catch specific error
4     handle(e)
5 except (TypeError, KeyError):  # multiple
6     handle()
7 else:
8     no_error_code()           # runs if NO exception
9 finally:
10    cleanup()                  # ALWAYS runs
```

Raise: `raise ValueError("msg")` — **Custom:** `class MyError(Exception): pass` **Common**

errors: `ValueError` `TypeError` `ZeroDivisionError` `IndexError` `KeyError` `FileNotFoundError`
`AttributeError`

6. OOP

PYTHON

```
1  class Child(Parent):                # inheritance
2      def __init__(self, a, b):
3          super().__init__(a)          # call parent ctor
4          self.__b = b                 # private
5          self._c = 0                 # protected
6
7      def __str__(self): return "... " # print() uses this
8      def method(self): ...
```

Encapsulation: public `self.x` · protected `self._x` · private `self.__x` **Polymorphism:** same method name, different class behavior **Abstract:** `from abc import ABC, abstractmethod` — `class X(ABC):` + `@abstractmethod def m(self): pass` **Interface:** abstract class with only abstract methods **Composition:** `self.obj = OtherClass()` (has-a) vs inheritance (is-a)

<code>__init__</code>	constructor	<code>super()</code>	parent methods	<code>isinstance(obj, Cls)</code>	type check
-----------------------	-------------	----------------------	----------------	-----------------------------------	------------

7. UNIT TESTING

PYTHON

```
1  import unittest
2  class TestX(unittest.TestCase):
3      def setUp(self): ...             # before EACH test
4      def tearDown(self): ...          # after EACH test
5      def test_method(self):
6          self.assertEqual(a, b)
7          self.assertTrue(x)
8          self.assertFalse(x)
9          self.assertIn(a, b)
10         self.assertIsNone(x)
11         self.assertIsInstance(x, Cls)
12         with self.assertRaises(Err): code()
13  if __name__ == "__main__": unittest.main()
```

Rule: test method names **must** start with `test_`

8. FILE I/O & JSON

PYTHON

```
1  with open("f.txt", "r") as f:    # auto-closes
2      content = f.read()           # whole file
3      lines = f.readlines()        # list of lines
4
5  with open("f.txt", "w") as f:
6      f.write("text")
7
8  import json
9  with open("f.json", "r") as f: data = json.load(f)    # read
10 with open("f.json", "w") as f: json.dump(data, f, indent=2) # write
```

Modes: `"r"` read · `"w"` write (overwrites) · `"a"` append **Safe read:** wrap in `try/except`
`FileNotFoundError`

9. STRING METHODS

Method	Result
<code>.lower()</code> <code>.upper()</code> <code>.strip()</code>	case & whitespace
<code>.split(sep)</code> <code>.replace(old,new)</code>	transform
<code>.find(sub)</code> <code>.count(sub)</code> <code>.startswith(s)</code> <code>.endswith(s)</code>	search
<code>.isalpha()</code> <code>.isdigit()</code> <code>.isalnum()</code>	validation
<code>", "</code> <code>.join(lst)</code>	join list to string

`ord("A")=65` · `chr(65)="A"` · `"A"<=ch<="Z"` · `"a"<=ch<="z"`

10. QUICK RECIPES

TEXT

- 1 Palindrome: `cleaned == cleaned[::-1]` (after `.lower()` + removing non-alnum)
- 2 Anagram: `sorted(w1.lower()) == sorted(w2.lower())`
- 3 Divisors: `[i for i in range(1,n+1) if n%i==0]`
- 4 Caesar: `chr((ord(ch)-base+shift)%26 + base)` (base=`ord("A")` or `ord("a")`)
- 5 Deep copy: `[i[:] for i in nested]`
- 6 Shallow: `lst.copy()` or `lst[:]`
- 7 Frequency: `d[k] = d.get(k, 0) + 1`
- 8 Sort by key: `sorted(lst, key=lambda x: x["field"])`

11. MUST-KNOW RULES

1. `self` = first param of every class method (not counted as argument)
2. `__init__` runs when creating object · `__str__` runs on `print(obj)`
3. `super().__init__()` in child → calls parent constructor
4. `@abstractmethod` → subclass MUST implement it · cannot instantiate ABC
5. `global x` = modify global var · `nonlocal x` = modify enclosing var
6. `with open()` = auto-close files · always prefer it
7. `try/else` = runs if no error · `finally` = always runs
8. Test methods must start with `test_` · `setUp()` before each · `tearDown()` after each
9. `raise Error("msg")` to throw · `except Error as e` to catch